Adaline

Podstawy AI · laboratorium

Funkcja straty i spadek gradientu

Przypomnienie: perceptron

$$z = \boldsymbol{w}^T \boldsymbol{x} + b, \quad \hat{y} = \begin{cases} 1 & \text{jeśli } z \geq 0 \\ 0 & \text{jeśli } z < 0 \end{cases}, \quad \Delta w_j = \eta(y - \hat{y}) x_j$$

Co zobaczyliśmy:

- zatrzymuje się na **pierwszej** prostej, która rozdziela klasy, nawet jeśli leży tuż przy punktach
- gdy klas nie da się rozdzielić, wagi zmieniają się **bez końca**
- η prawie nie ma znaczenia

> Wspólna przyczyna: błąd $y - \hat{y}$ ma tylko wartości $0, \pm 1$.

Czego perceptron nie widzi

Dwa punkty klasy 1, oba sklasyfikowane źle ($\hat{y} = 0$):

Ledwo źle: $z = -0.01$
błąd $y - \hat{y} = 1$

Bardzo źle: $z = -50$
błąd $y - \hat{y} = 1$

I punkt **ledwo dobrze** ($z = 0.01$): błąd 0, zero poprawki.

> Perceptron nie wie, **jak bardzo** się myli. Pomysł: liczyć błąd z z, a nie z $\hat{y}$.

Część 1

Adaline

Adaline: ADAptive LInear NEuron

- Widrow i Hoff (jego doktorant), Stanford, 1960, kilka lat po perceptronie
- Ten sam neuron: $z = \boldsymbol{w}^T \boldsymbol{x} + b$
- Różnica: do uczenia używamy **liniowej funkcji aktywacji**

$$\sigma(z) = z$$

  czyli po prostu samego z (funkcja tożsamościowa)
- Funkcja skokowa zostaje, ale **tylko do końcowej predykcji**

> Reguła Adaline nazywana jest też **regułą Widrowa–Hoffa**.

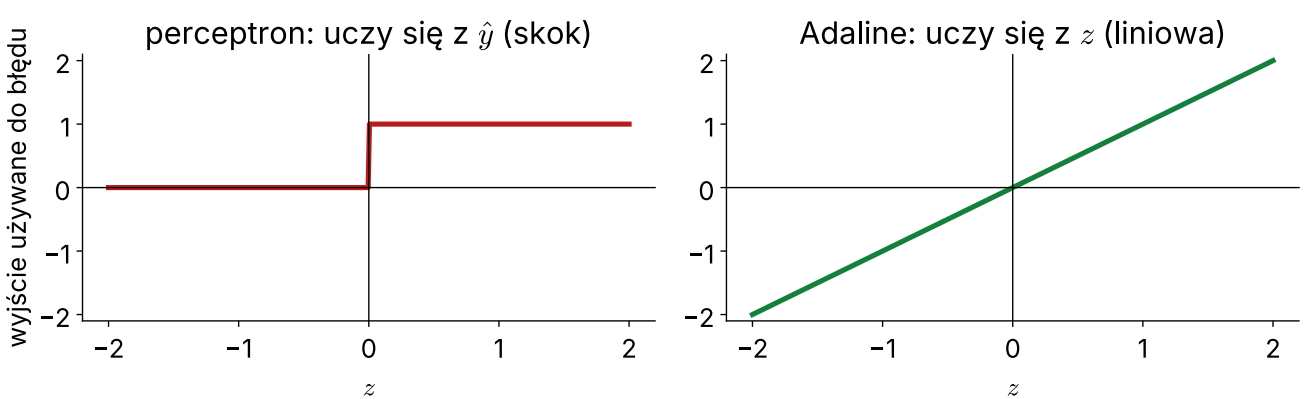
Perceptron i Adaline: jedno miejsce różnicy

Perceptron: błąd liczony **po** funkcji skokowej

$$\boxed{\boldsymbol{x}} \rightarrow \boxed{z = \boldsymbol{w}^T \boldsymbol{x} + b} \rightarrow \boxed{\text{skok: } \hat{y} \in \{0, 1\}} \rightarrow \text{błąd } y - \hat{y}$$

Adaline: błąd liczony **przed** funkcją skokową

$$\boxed{\boldsymbol{x}} \rightarrow \boxed{z = \boldsymbol{w}^T \boldsymbol{x} + b} \rightarrow \boxed{\sigma(z) = z \in \mathbb{R}} \rightarrow \text{błąd } y - z$$



Predykcja w Adaline

- Etykiety to 0 i 1, a z może być dowolną liczbą
- Uczymy tak, żeby z było **blisko** etykiety: blisko 0 dla klasy 0, blisko 1 dla klasy 1
- Klasę przewidujemy progiem w połowie drogi:

$$\hat{y} = \begin{cases} 1 & \text{jeśli } z \geq 0.5 \\ 0 & \text{w przeciwnym razie} \end{cases}$$

> W uczeniu: ciągłe z. W predykcji: próg 0.5.

Funkcja straty: jak źle jest?

Średni błąd kwadratowy (MSE, *mean squared error*) na całym zbiorze:

$$L(\boldsymbol{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - z^{(i)} \right)^2$$

- Dla każdego przykładu: błąd $e^{(i)} = y^{(i)} - z^{(i)}$, podnosimy do kwadratu
- Uśredniamy po wszystkich n przykładach
- L zależy od wag: inne $\boldsymbol{w}, b$ to inne z, a więc inna strata

Uczenie = szukanie w i b, dla których L jest najmniejsze.

Przykład: strata na punktach z kartki

Punkty A–D z poprzednich zajęć, wagi startowe $w_1 = w_2 = b = 0$, więc $z = 0$ dla każdego:

pkt	y	z	$e = y - z$	e^2
A	1	0	1	1
B	0	0	0	0
C	1	0	1	1
D	0	0	0	0

$$L = \frac{1}{4}(1 + 0 + 1 + 0) = 0.5$$

Dlaczego kwadrat?

- Znak błędu nie ma znaczenia: -2 i $+2$ są tak samo złe
- Duże błędy liczą się **bardziej** niż małe
- Funkcja jest **gładka**: ma pochodną w każdym punkcie
- Jest **wypukła**: ma jedno minimum, bez „fałszywych dołków”

Funkcja skokowa nie ma użytecznej pochodnej: jest płaska wszędzie poza skokiem. Dlatego perceptronu nie da się uczyć spadkiem gradientu.

Część 2

Spadek gradientu

Idea: schodzenie z góry we mgle

- Stoisz na zboczu, widzisz tylko nachylenie **pod stopami**
- Robisz krok **w dół**, czyli w stronę przeciwną do nachylenia
- Powtarzasz, aż teren zrobi się płaski (dno doliny)

Dla jednej wagi, krok numer k:

$$w_{k+1} = w_k - \eta \frac{dL}{dw}(w_k) \qquad (\text{krócej: } \; w := w - \eta \frac{dL}{dw} \;)$$

- $\frac{dL}{dw}(w_k)$ — **pochodna** policzona w obecnym punkcie w_k: nachylenie
- η — **współczynnik uczenia**: jak długi krok robimy

Przykład z jedną wagą

$$L(w) = (w-3)^2, \qquad \frac{dL}{dw} = 2(w-3)$$

Minimum jest w $w = 3$. Co mówi pochodna?

$w_k = 0$ (na lewo od minimum):
$\frac{dL}{dw} = 2(0-3) = -6 < 0$
$w_{k+1} = 0 - \eta \cdot (-6) = 6\eta$
w rośnie ✓

$w_k = 5$ (na prawo od minimum):
$\frac{dL}{dw} = 2(5-3) = 4 > 0$
$w_{k+1} = 5 - \eta \cdot 4$
w maleje ✓

> Znak pochodnej sam mówi, w którą stronę iść. Nie musimy wiedzieć, gdzie jest minimum.

Kartka 1: spadek gradientu ręcznie

$$L(w) = (w-3)^2, \quad w_{k+1} = w_k - \eta \cdot 2(w_k - 3), \quad \text{start: } w_0 = 0$$

Każdy krok liczysz z **poprzedniego**: $w_0 \to w_1 \to w_2 \to w_3$. Np. dla $\eta = 0.25$:

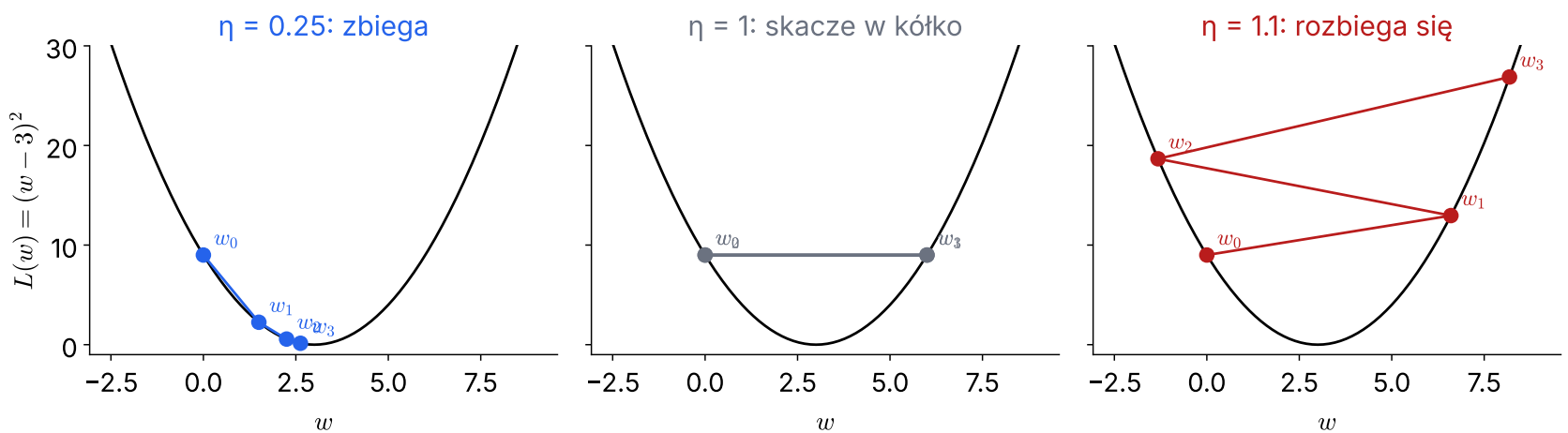
$$w_1 = w_0 - 0.25 \cdot 2(w_0 - 3) = 0 - 0.25 \cdot (-6) = 1.5$$

Policz w_1, w_2, w_3 i $L(w_k)$ dla trzech wartości η. **Co się dzieje? Narysuj kroki na paraboli.**

η	w_0	w_1	w_2	w_3
0.25	0	?	?	?
1	0	?	?	?
1.1	0	?	?	?

Rozwiązanie

η	w_0	w_1	w_2	w_3	$L(w_3)$
0.25	0	1.5			
1	0				
1.1	0				gm



Część 3

Reguła uczenia Adaline

Wiele wag: gradient

- Strata zależy od **wszystkich** wag i biasu: $L(w_1, w_2, ..., w_m, b)$
- Dla każdej wagi liczymy osobno nachylenie: **pochodną cząstkową** $\frac{\partial L}{\partial w_j}$
  (pochodna po w_j, pozostałe wagi traktujemy jak stałe)
- Wektor wszystkich pochodnych to **gradient** ∇L
- Krok spadku gradientu, dla każdej wagi naraz:

$$\Delta w_j = -\eta \frac{\partial L}{\partial w_j}, \qquad \Delta b = -\eta \frac{\partial L}{\partial b}$$

Pochodna MSE krok po kroku (1/2)

$$L = \frac{1}{n}\sum_i \left(e^{(i)}\right)^2, \quad \text{gdzie} \quad e^{(i)} = y^{(i)} - z^{(i)}, \quad z^{(i)} = \sum_j w_j x_j^{(i)} + b$$

Krok 1. Pochodna sumy to suma pochodnych:

$$\frac{\partial L}{\partial w_j} = \frac{1}{n}\sum_i \frac{\partial}{\partial w_j}\left(e^{(i)}\right)^2$$

Krok 2. Reguła łańcuchowa: pochodna kwadratu razy pochodna środka

$$\frac{\partial}{\partial w_j}\left(e^{(i)}\right)^2 = 2e^{(i)} \cdot \frac{\partial e^{(i)}}{\partial w_j}$$

Pochodna MSE krok po kroku (2/2)

Krok 3. Jak $e^{(i)} = y^{(i)} - \left(w_1 x_1^{(i)} + \ldots + w_j x_j^{(i)} + \ldots + b\right)$ zależy od w_j?

y nie zależy od wag, a z sumy zostaje tylko składnik z w_j:

$$\frac{\partial e^{(i)}}{\partial w_j} = -x_j^{(i)}, \qquad \frac{\partial e^{(i)}}{\partial b} = -1$$

Krok 4. Składamy:

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n}\sum_i e^{(i)} x_j^{(i)}, \qquad \frac{\partial L}{\partial b} = -\frac{2}{n}\sum_i e^{(i)}$$

Reguła Adaline

Wstawiamy do $\Delta w_j = -\eta \frac{\partial L}{\partial w_j}$, minusy się znoszą:

$$\Delta w_j = \eta \cdot \frac{2}{n} \sum_i \left(y^{(i)} - z^{(i)}\right) x_j^{(i)}, \qquad \Delta b = \eta \cdot \frac{2}{n} \sum_i \left(y^{(i)} - z^{(i)}\right)$$

	Perceptron	**Adaline**
poprawka	$\eta(y - \hat{y})x_j$	$\eta \cdot \frac{2}{n} \sum_i (y - z)x_j$
błąd	0 lub ± 1	dowolna liczba
kiedy	po każdym przykładzie	raz na epokę, z całego zbioru

W kodzie: jedna linijka zamiast pętli

Suma $\sum_i e^{(i)} x_j^{(i)}$ dla wszystkich j naraz to mnożenie macierzy przez wektor:

```
errors = y - self.net_input(X)                  # wektor e, n elementów
self.w_ += self.eta * 2.0 * X.T @ errors / X.shape[0]
self.b_ += self.eta * 2.0 * errors.mean()
```

Przykład mnożenia: $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \begin{pmatrix} 7 \\ 8 \\ 9 \end{pmatrix} = \begin{pmatrix} 1 \cdot 7 + 2 \cdot 8 + 3 \cdot 9 \\ 4 \cdot 7 + 5 \cdot 8 + 6 \cdot 9 \end{pmatrix} = \begin{pmatrix} 50 \\ 122 \end{pmatrix}$

Część 4

Adaline w kodzie: η i epoki

Co robicie w notebooku

1. Kartka 1 w kodzie: trajektorie spadku gradientu dla różnych η
2. Uzupełniacie **4 linijki** w `fit`
3. Iris: za duże i za małe η, szukanie **granicy stabilności**
4. **Mapa η × epoki**: kiedy więcej epok pomaga, a kiedy nie
5. **Granica decyzyjna w czasie**: jak przesuwa się prosta

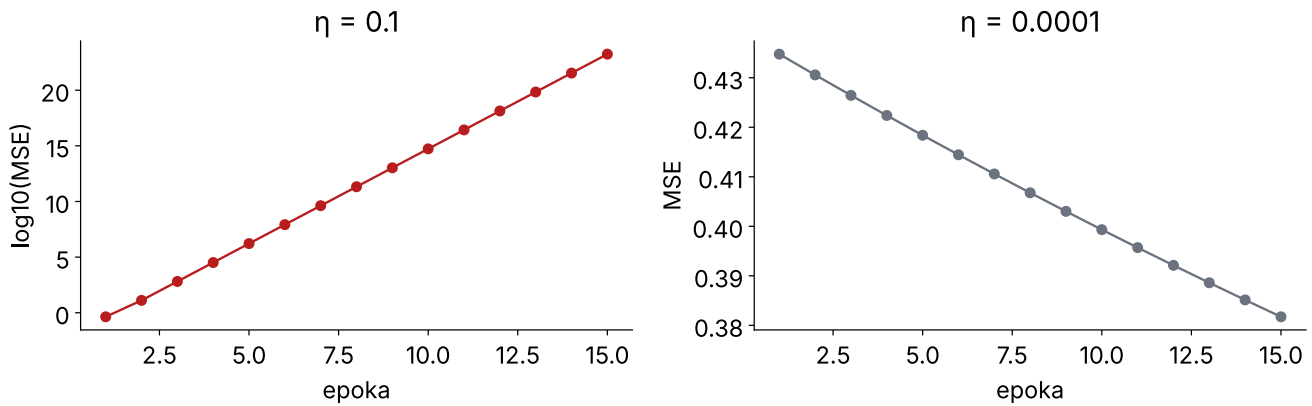
Eksperyment: przewidź, zanim uruchomisz

Adaline na Iris (setosa i versicolor, dane w centymetrach), 15 epok.

Jak będzie wyglądała strata dla $\eta = 0.1$, a jak dla $\eta = 0.0001$?

> Przypomnij sobie kartkę 1. Na perceptronie η nie miało znaczenia. A tutaj?

Wynik: dwa rodzaje kłopotów



- $\eta = 0.1$: strata **rośnie** do 10^{23} po 15 epokach (przeskakujemy minimum)
- $\eta = 0.0001$: strata maleje, ale **bardzo wolno** ($0.435 \longrightarrow 0.382$)

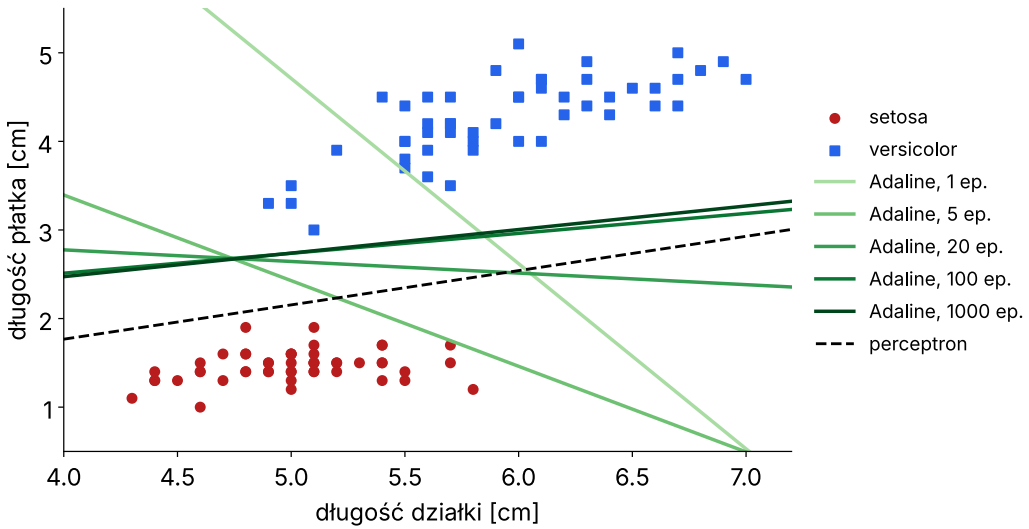
η razy liczba epok

MSE / trafność po różnej liczbie epok:

η	15 epok	100 epok	1000 epok
0.001	0.193 / 74%	0.120 / 100%	0.014 / 100%
0.005	0.135 / 100%	0.031 / 100%	0.012 / 100%
0.01	0.102 / 100%	0.014 / 100%	0.012 / 100%
0.02	0.060 / 100%	0.012 / 100%	0.012 / 100%
0.025	0.55 / 50%	32 / 50%	10^{20} / 50%

- Mniejsze η = więcej epok: $\eta = 0.001$ po 1000 epokach $\approx \eta = 0.01$ po 100
- Strata ma **dno** (ok. 0.012): dalej więcej epok nic nie da
- Powyżej granicy stabilności więcej epok tylko **pogarsza** sprawę

Trafność to nie wszystko



$\eta = 0.01$:

- po 5 epokach: 100% trafień
- a prosta **dalej się przesuwa** aż do ok. 1000 epok
- perceptron zatrzymałby się na pierwszej prostej, która rozdziela klasy

> Adaline nie pyta „czy się mylę?”, tylko „czy mogę zmniejszyć stratę?”.

Czego Adaline nadal nie potrafi

- Granica decyzyjna to wciąż **prosta**: $z = 0.5$
- XOR: najlepsza prosta w sensie MSE daje $z \approx 0.5$ dla **wszystkich** 4 punktów, czyli model nic nie wie
- Ale na danych nierozdzielnych (versicolor/virginica) Adaline **zbiega** do najlepszej prostej, zamiast skakać bez końca

> Następny krok: zamiast $\sigma(z) = z$ **sigmoida** (regresja logistyczna), a potem neurony w **warstwach**. Uczymy je tak samo: funkcja straty i spadek gradientu.

Podsumowanie

- Adaline liczy błąd z **ciągłego** z, a nie z $\hat{y}$
- **Funkcja straty** (MSE) mówi jedną liczbą, jak źle jest
- **Spadek gradientu**: $w_{k+1} = w_k - \eta \cdot$ pochodna
- Za małe η: potrzeba wielu epok. Za duże: rozbieżność, i epoki nie pomogą
- 100% trafień nie znaczy, że uczenie się skończyło

> Następnym razem: **standaryzacja** i **stochastyczny spadek gradientu**, czyli jak uczyć szybciej i stabilniej.

Dodatek

Kartka 2: jeden krok Adaline

Punkty A–D, start $w_1 = w_2 = b = 0$, $\eta = 0.05$, $n = 4$.

1. Policz z i $e = y - z$ dla każdego punktu (już to znamy: $e = 1, 0, 1, 0$)
2. Policz sumy: $\sum ex_1$, $\sum ex_2$, $\sum e$
3. Policz $\Delta w_1, \Delta w_2, \Delta b$ ze wzoru $\Delta w_j = \eta \cdot \frac{2}{n} \cdot \sum ex_j$
4. **Nowe** wagi: policz z, e i predykcję ($z \geq 0.5$) dla każdego punktu
5. Policz nową stratę L. Zmalała?

Rozwiązanie

$\sum ex_1 = 1 \cdot 2 + 1 \cdot 3 = 5, \quad \sum ex_2 = 1 \cdot 1 + 1 \cdot 2 = 3, \quad \sum e = 2, \quad \eta \cdot \frac{2}{n} = 0.025$
$\Delta w_1 = 0.025 \cdot 5 = 0.125, \quad \Delta w_2 = 0.025 \cdot 3 = 0.075, \quad \Delta b = 0.025 \cdot 2 = 0.05$

pkt	y	nowe z	$e = y - z$	e^2	$\hat{y}$
A	1	0.375	0.625	0.391	0
B	0	0.400	−0.400	0.160	0
C	1	0.575	0.425	0.181	1
D	0	0.200	−0.200	0.040	0

L: $0.5 \rightarrow \frac{0.391+0.160+0.181+0.040}{4} = 0.193$ **zmalała** ✓